

Simulador de Scheduling: Análisis Comparativo de FCFS y Round Robin

Isaac Felipe Morún Moreira

Universidad Invenio — Licenciatura en Tecnologías de Información y Comunicación Empresarial

TIIT2007 – Sistemas Operativos | Julio 2026

1. Resumen

Este documento presenta el diseño, la implementación y la evaluación experimental de un simulador de planificación de CPU desarrollado en C++17 para el Proyecto 1 del curso TIIT2007. El simulador modela dos algoritmos de planificación de corto plazo —First Come, First Served (FCFS) y Round Robin (RR) con quantum configurable— sobre cargas de trabajo sintéticas leídas desde archivos de texto en formato id,llegada,rafaga, y reporta tiempo de espera, tiempo de retorno y porcentaje de utilización de CPU, tanto por proceso como en promedio, junto con un diagrama de Gantt en representación ASCII. Se evaluaron dos conjuntos de datos: uno de 10 procesos, correspondiente al requisito mínimo del enunciado, y una ampliación voluntaria de 50 procesos generada con semilla fija 20260729 para garantizar reproducibilidad. En el dataset de 10 procesos, FCFS obtuvo un tiempo de espera promedio de 17.40 u.t., 21.27 % menor que Round Robin con quantum 2 (22.10 u.t.) y 15.94 % menor que con quantum 4 (20.70 u.t.); en el dataset de 50 procesos la brecha se amplió a 28.27 % y 26.86 % respectivamente. La utilización de CPU resultó idéntica entre los tres escenarios de cada dataset (100.00 % y 99.78 %), resultado que se explica formalmente por la propiedad de conservación de trabajo de ambos algoritmos. Round Robin, en contrapartida, redujo la desviación estándar del tiempo de espera un 15.50 % y benefició a los procesos de ráfaga corta en un 34.52 %.

Palabras clave—planificación de procesos, FCFS, Round Robin, quantum, tiempo de espera, C++17.

2. Introducción

2.1 Contexto del problema

La planificación de procesos es el mecanismo mediante el cual un sistema operativo multiprogramado decide cuál de los procesos en estado listo obtiene la CPU en cada instante. Esta decisión, tomada por el planificador de corto plazo, condiciona directamente el comportamiento observable del sistema: determina cuánto espera cada proceso antes de progresar, cuánto tarda en completarse desde que llega, y qué fracción del tiempo la CPU permanece productiva [1]. Dado que un mismo conjunto de procesos puede producir métricas muy distintas según el algoritmo aplicado, comparar planificadores bajo una carga controlada es el procedimiento estándar para razonar sobre sus compromisos de diseño.

El problema abordado en este proyecto es que dicha comparación resulta difícil de realizar sobre un sistema operativo real, donde intervienen simultáneamente operaciones de entrada/salida, interrupciones, prioridades

dinámicas y contención de memoria. Un simulador de eventos discretos aísla la variable de interés —la política de planificación— y permite observar su efecto de forma reproducible.

2.2 Objetivo general

Implementar en C++17 un simulador de planificación de CPU que modele los algoritmos FCFS y Round Robin sobre una misma carga de procesos sintética, y comparar cuantitativamente su comportamiento en términos de tiempo de espera, tiempo de retorno y utilización de CPU.

2.3 Objetivos específicos

- 1) Diseñar una estructura de datos que separe los atributos de entrada de un proceso de las métricas calculadas durante la simulación.
- 2) Implementar FCFS con ordenamiento estricto por tiempo de llegada y sin apropiación.
- 3) Implementar Round Robin apropiativo con quantum configurable por línea de comandos.
- 4) Calcular tiempo de espera y tiempo de retorno por proceso y en promedio para ambos algoritmos.
- 5) Calcular el porcentaje de utilización de CPU a partir de la traza de segmentos de ejecución.
- 6) Generar un diagrama de Gantt en representación ASCII que refleje el orden real de ejecución.
- 7) Validar el lector de entrada frente a archivos con líneas deliberadamente malformadas.
- 8) Comparar experimentalmente ambos algoritmos con al menos dos valores de quantum sobre dos datasets de tamaño distinto.

2.4 Alcance y limitaciones

El simulador modela exclusivamente planificación de CPU a nivel de procesos, bajo el supuesto de que la ráfaga de cada proceso es conocida de antemano, fija y sin interrupciones de entrada/salida. Se implementaron dos algoritmos: FCFS y Round Robin.

Quedan explícitamente fuera del alcance: Shortest Job First, planificación por prioridades, colas multinivel con retroalimentación, planificación de hilos, y el modelado del costo del cambio de contexto, que se asume nulo. El simulador tampoco mide el tiempo de respuesta —el intervalo entre la llegada de un proceso y su primera asignación de CPU—, que es precisamente la métrica en la que Round Robin exhibe su ventaja teórica; esta omisión se discute en la Sección 8.3. La interfaz es de consola, sin componentes gráficos.

3. Marco Teórico

3.1 Planificación de CPU y métricas de evaluación

En un sistema multiprogramado, varios procesos residen simultáneamente en memoria y compiten por la CPU. Cuando la CPU queda disponible, el planificador de corto plazo selecciona un proceso de la cola de listos y le transfiere el control mediante el despachador [1]. Un planificador se denomina no apropiativo cuando el proceso conserva la CPU hasta terminar su ráfaga o bloquearse voluntariamente, y apropiativo cuando el sistema puede retirarle la CPU antes de que termine.

Las métricas empleadas en este trabajo son las convencionales de la literatura [1], [2]. El tiempo de retorno de un proceso es el intervalo entre su llegada y su finalización. El tiempo de espera es la porción de ese intervalo durante la cual el proceso estuvo listo pero sin CPU, es decir, tiempo de retorno menos ráfaga total. La utilización de CPU es el porcentaje del intervalo total de simulación en que la CPU no estuvo ociosa. Una propiedad relevante para la Sección 8 es que ambos algoritmos aquí evaluados son conservadores de trabajo: nunca dejan la CPU ociosa si hay al menos un proceso listo.

3.2 First-Come, First-Served (FCFS)

FCFS es el algoritmo de planificación más simple: los procesos reciben la CPU en el orden estricto en que llegan a la cola de listos, y una vez asignada la conservan hasta completar íntegramente su ráfaga [1]. Su implementación natural es una cola FIFO y su costo de decisión es constante.

Su desventaja característica es el efecto convoy: si un proceso de ráfaga larga ocupa la CPU, todos los procesos cortos que llegaron después quedan bloqueados detrás de él, elevando el tiempo de espera promedio del conjunto de forma desproporcionada [1], [2]. La severidad del efecto depende del orden de llegada y de la dispersión de las ráfagas; en cargas donde las ráfagas son relativamente homogéneas, FCFS puede resultar competitivo o incluso superior en tiempo de espera promedio.

3.3 Round Robin (RR)

Round Robin fue concebido para sistemas de tiempo compartido. Asigna a cada proceso una porción fija de CPU denominada quantum; si el proceso no completa su ráfaga dentro de ese intervalo, es apropiado y reencolado al final de la cola de listos [1], [3]. El resultado es un reparto cíclico de turnos que acota el tiempo que cualquier proceso puede esperar antes de recibir atención.

El tamaño del quantum es el parámetro crítico del algoritmo. Con un quantum muy pequeño, el reparto tiende a un procesador compartido ideal, pero el número de cambios de contexto crece y, en un sistema real, su costo llega a dominar el tiempo útil. Con un quantum mayor que la ráfaga máxima del sistema, Round Robin degenera exactamente en FCFS [2], [3]. La consecuencia analítica es que Round Robin no minimiza el tiempo de espera promedio —ese es un resultado de Shortest Job First—, sino que reduce su dispersión entre procesos y mejora el tiempo de respuesta.

3.4 Relación con la observación práctica en xv6

El kernel didáctico xv6, utilizado como referencia práctica del curso, implementa una variante de planificación round-robin en su función scheduler() del archivo proc.c: recorre cíclicamente la tabla fija de procesos buscando el siguiente en estado RUNNABLE y le cede la CPU hasta que el temporizador dispara la apropiación. La diferencia estructural con este simulador es que xv6 recorre un arreglo estático, mientras que aquí se mantiene una cola explícita `std::queue<int>` de índices. La cola explícita permite representar de forma directa dos eventos que en el arreglo quedan implícitos: la incorporación de un proceso recién llegado y el reingreso de un proceso apropiado, que es exactamente el punto donde se define la regla de desempate descrita en la Sección 4.3.

4. Diseño de la Solución

4.1 Arquitectura general

El simulador se organiza en cuatro capas con responsabilidades disjuntas, coordinadas por `main.cpp`:

- **Capa de datos:** `Proceso.h` y `Simulacion.h` definen las estructuras `Proceso`, `Segmento` y `ResultadoSimulacion`.
- **Capa de entrada:** `Lector.h/cpp` lee y valida el archivo de procesos.
- **Capa de algoritmos:** `FCFS.h/cpp` y `RoundRobin.h/cpp` implementan cada política de planificación.
- **Capa de reporte:** `Metricas.h/cpp` y `Gantt.h/cpp` transforman el resultado en salida legible.

La restricción de diseño que sostiene esta separación es que ambos algoritmos devuelven el mismo tipo, `ResultadoSimulacion`. En consecuencia, `Metricas` y `Gantt` operan sin conocer qué algoritmo produjo el resultado, y agregar un tercer algoritmo no requiere modificar ninguna línea de la capa de reporte.

4.2 Estructuras de datos y justificación

La estructura `Proceso` separa deliberadamente los campos de entrada —id, llegada y ráfaga, que no se modifican tras la lectura— de los campos de trabajo —restante, finalizacion, tiempoEspera y tiempoRetorno. El campo restante es una copia de trabajo de ráfaga que Round Robin decreuenta en cada turno; preservar ráfaga intacta es lo que permite calcular el tiempo de espera como `tiempoRetorno` menos ráfaga al final de la simulación, sin necesidad de acumular esperas parciales turno a turno.

`std::queue<int>` para la cola de listos. Round Robin requiere exactamente dos operaciones sobre la cola: extraer el elemento más antiguo y anexas al final. `std::queue`, adaptador sobre `std::deque`, ofrece ambas en tiempo constante amortizado. Se descartaron dos alternativas: `std::vector` con borrado en la posición cero, que degrada la extracción a $O(n)$ por desplazamiento de elementos, y `std::priority_queue`, cuya semántica de orden por clave es innecesaria aquí —Round Robin no ordena, rota— e introduciría un costo $O(\log n)$ sin beneficio. La cola almacena índices `int` dentro del vector de procesos, no copias de la

estructura, de modo que las modificaciones de restante son visibles para todos los turnos del mismo proceso.

std::vector<Segmento> para la traza de ejecución. Registrar la línea de tiempo como una secuencia de tramos {idProceso, inicio, fin}, con idProceso igual a -1 representando CPU ociosa, permite derivar tanto el diagrama de Gantt como la utilización de CPU de una misma fuente. La utilización se calcula en un único recorrido lineal sobre los segmentos, sin necesidad de mantener contadores paralelos durante la simulación.

4.3 Supuestos de diseño

- **Regla de desempate en Round Robin.** Cuando un proceso llega en el mismo instante en que otro agota su quantum sin terminar, el proceso recién llegado se encola antes que el apropiado. En RoundRobin.cpp esto se materializa invocando encolarLlegadasHasta(tiempoActual) antes de reencolar el proceso actual. La convención respeta el orden real de los eventos y es la adoptada por la bibliografía de referencia [1].
- **Ordenamiento determinista.** Ambos algoritmos ordenan por tiempo de llegada y, en caso de empate, por identificador ascendente, garantizando que dos ejecuciones sobre el mismo archivo produzcan resultados idénticos.
- **Tiempo ocioso explícito.** Cuando la cola de listos está vacía y aún no ha llegado ningún proceso, se registra un segmento con idProceso igual a -1 en lugar de avanzar el reloj silenciosamente. Esta decisión es lo que hace medible la utilización de CPU.
- **Costo de cambio de contexto nulo** y ausencia de operaciones de entrada/salida durante la ráfaga.
- **Tolerancia a fallos en la entrada.** Una línea inválida no aborta la lectura: se reporta por stderr y se descarta, de modo que un archivo parcialmente corrupto sigue produciendo una simulación válida sobre las líneas correctas.

5. Implementación

5.1 Lenguaje y estándar

El simulador está escrito en C++17 y utiliza exclusivamente la biblioteca estándar, sin dependencias externas. El entorno oficial de desarrollo y prueba fue g++ 15.2.0 sobre Ubuntu 26.04 LTS ejecutado en VirtualBox, con las banderas -std=c++17 -Wall. El código base consta de 363 líneas distribuidas en 13 archivos fuente.

5.2 Módulos

La Tabla 1 resume la responsabilidad de cada archivo fuente.

Tabla 1: Módulos del simulador y su responsabilidad.

Archivo	Responsabilidad
Proceso.h	Estructura de datos de un proceso
Simulacion.h	Tipos compartidos: Segmento y ResultadoSimulacion
Lector.h.cpp	Lectura y validación de la entrada
FCFS.h.cpp	Algoritmo FCFS, sin apropiación

Archivo	Responsabilidad
RoundRobin.h.cpp	Round Robin con quantum
Metricas.h.cpp	Promedios y utilización de CPU
Gantt.h.cpp	Diagrama de Gantt ASCII
main.cpp	Orquestación y argumentos

El lector distingue tres clases de error, reportadas con el número de línea: campos faltantes, campos no numéricos, y valores semánticamente inválidos (llegada negativa o ráfaga menor o igual a cero). La apertura fallida del archivo se señala mediante std::runtime_error, capturada en main.cpp.

El núcleo de Round Robin concentra la decisión de diseño descrita en la Sección 4.3: primero se encolan las llegadas ocurridas durante el quantum, y solo después se reencola el proceso apropiado.

El generador de Gantt construye dos cadenas paralelas —la barra de etiquetas y la regla de tiempos— calculando el relleno de cada marca a partir del ancho de la etiqueta correspondiente, de modo que ambas líneas quedan alineadas independientemente del número de dígitos de los instantes.

5.3 Compilación y ejecución

```
make
./simulador data/procesos.txt 2
./simulador data/procesos.txt 4
./simulador data/procesos50.txt 2
./simulador data/procesos50.txt 4
```

Compilación manual equivalente:

```
g++ -std=c++17 -Wall -Isrc \
-o simulador src/*.cpp
```

Cada ejecución corre ambos algoritmos sobre el mismo dataset e imprime, para cada uno, el diagrama de Gantt, la tabla de tiempos por proceso, los promedios y la utilización de CPU.

6. Resultados Experimentales

Todos los valores de tiempo se expresan en unidades de tiempo abstractas (u.t.), conforme a la convención habitual en simulación de planificación de eventos discretos [1].

6.1 Entorno de prueba

Máquina virtual Ubuntu 26.04 LTS sobre VirtualBox, compilador g++ 15.2.0, banderas -std=c++17 -Wall. El simulador modela tiempo lógico de eventos discretos, por lo que las métricas de espera, retorno y utilización son deterministas y no dependen del hardware anfitrión. Como referencia de rendimiento se midió adicionalmente el tiempo real de ejecución del binario: 0.135 s para el dataset de 10 procesos y 0.077 s para el de 50, valores dominados por el arranque del proceso y no por el cálculo.

6.2 Dataset de referencia (10 procesos)

El dataset mínimo exigido por el enunciado contiene 10 procesos con llegadas entre 0 y 9 u.t. y ráfagas entre 2 y 8 u.t. (Tabla 2). La suma de ráfagas es 45 u.t. La Tabla 3 detalla los tiempos por proceso obtenidos con cada algoritmo y la Tabla 4 resume las métricas agregadas.

Tabla 2: Carga de trabajo del dataset de referencia.

Proc.	Lleg.	Ráf.	Proc.	Lleg.	Ráf.
P1	0	5	P6	5	4
P2	1	3	P7	6	7

Proc.	Lleg.	Ráf.	Proc.	Lleg.	Ráf.
P3	2	8	P8	7	3
P4	3	6	P9	8	5
P5	4	2	P10	9	2

Tabla 3: Tiempos de espera (E) y retorno (R) por proceso, dataset de referencia.

Proc.	FCFS E	FCFS R	q=2 E	q=2 R	q=4 E	q=4 R
P1	0	5	19	24	13	18
P2	4	7	9	12	3	6
P3	6	14	33	41	29	37
P4	13	19	30	36	32	38
P5	18	20	6	8	11	13
P6	19	23	21	25	13	17
P7	22	29	32	39	31	38
P8	28	31	25	28	19	22
P9	30	35	31	36	32	37
P10	34	36	15	17	24	26
Prom.	17.40	21.90	22.10	26.60	20.70	25.20

Tabla 4: Métricas agregadas, dataset de referencia (10 procesos).

Métrica	FCFS	q=2	q=4
T. espera promedio	17.40	22.10	20.70
T. retorno promedio	21.90	26.60	25.20
Desv. est. de la espera	10.97	9.27	9.85
Espera máxima	34	33	32
Turnos de CPU asignados	10	25	15
Utilización de CPU (%)	100.00	100.00	100.00

6.3 Dataset extendido (50 procesos)

Ampliación voluntaria no exigida por el enunciado, generada con semilla fija 20260729: llegadas acumuladas con saltos de 0 a 3 u.t. (rango 1–85) y ráfagas uniformes entre 1 y 15 u.t., con suma total de 462 u.t. y ráfaga media de 9.24 u.t. La Tabla 5 presenta las métricas agregadas.

Tabla 5: Métricas agregadas, dataset extendido (50 procesos).

Métrica	FCFS	q=2	q=4
T. espera promedio	180.72	251.96	247.10
T. retorno promedio	189.96	261.20	256.34
Turnos de CPU asignados	50	246	136
Utilización de CPU (%)	99.78	99.78	99.78

Dado el volumen de datos, el detalle por proceso se encuentra versionado en results/simulacion_50procesos.txt.

6.4 Diagrama de Gantt

La Figura 1 muestra la línea de tiempo generada por el simulador para FCFS sobre el dataset de referencia. Los diagramas de Round Robin, de mayor extensión por el número de turnos (25 con q=2 y 15 con q=4), están versionados en results/simulacion_completa.txt.



Figura 1: Diagrama de Gantt — FCFS, dataset de referencia.

6.5 Comparación cuantitativa

En ambos datasets, FCFS obtuvo el menor tiempo de espera promedio. En el dataset de referencia, la ventaja de FCFS sobre Round Robin fue de 21.27 % frente a q=2 y 15.94 % frente a q=4; en el dataset extendido, la brecha se amplió a 28.27 % y 26.86 % respectivamente. En tiempo de retorno la

ventaja fue de 17.67 % y 13.10 % (10 procesos) y de 27.27 % y 25.90 % (50 procesos).

Al aumentar el quantum de 2 a 4, el tiempo de espera de Round Robin se redujo 6.33 % en el dataset de referencia y 1.93 % en el extendido, acercándose progresivamente a FCFS, y el número de turnos de CPU cayó de 25 a 15 (−40.00 %) y de 246 a 136 (−44.72 %) respectivamente.

En sentido inverso, Round Robin con q=2 redujo la desviación estándar del tiempo de espera de 10.97 a 9.27 u.t. (−15.50 %) y la espera máxima de 34 a 33 u.t. Desagregando por longitud de ráfaga en el dataset de referencia, los cuatro procesos de ráfaga corta (menor o igual a 3 u.t.) pasaron de una espera media de 21.00 u.t. bajo FCFS a 13.75 u.t. bajo RR q=2, una mejora de 34.52 %, mientras que los tres procesos de ráfaga larga (mayor o igual a 6 u.t.) se degradaron de 13.67 a 31.67 u.t., un aumento de 131.71 %.

La utilización de CPU resultó idéntica entre los tres escenarios de cada dataset: 100.00 % con 10 procesos y 99.78 % con 50. Esta invariancia se analiza en la Sección 8.2.

7. Evaluación ISO/IEC 25010

Autoevaluación conforme a la rúbrica institucional (Excelente = 4, Competente = 3, En desarrollo = 2, Insuficiente = 1), con evidencia verificable en el repositorio. La Tabla 6 recoge las cinco características evaluadas.

Tabla 6: Autoevaluación ISO/IEC 25010.

Característica	Cal.	Justificación
Rendimiento	4	Ambos algoritmos son $O(n \log n)$ por el ordenamiento inicial; la simulación de RR es $O(\text{suma de ráfagas} / q)$. Tiempo real medido: 0.135 s (10 procesos) y 0.077 s (50). Se comparó el efecto del quantum sobre espera, retorno y número de turnos en dos datasets (Tablas 4 y 5).
Fiabilidad	4	Lector.cpp maneja cuatro condiciones de error: archivo inexistente (std::runtime_error), campos faltantes, campos no numéricos y valores semánticamente inválidos. Verificado con data/procesos_errores.txt, que descarta tres líneas inválidas y simula correctamente con las dos restantes. El dataset extendido usa semilla fija, garantizando reproducibilidad.
Usabilidad	3	La interfaz de consola valida el número de argumentos e imprime un mensaje de uso con código de salida 1. No obstante, no ofrece ayuda extendida, no valida que el quantum sea positivo antes de usarlo, y std::stoi sobre un quantum no numérico lanza una excepción no capturada. Es funcional pero mínima.
Compatibilidad	3	El código emplea únicamente la biblioteca estándar de C++17, sin llamadas específicas de plataforma ni dependencias externas. Compila sin advertencias bajo -Wall. La compatibilidad con Windows se argumenta por ausencia de dependencias, pero no fue verificada directamente en esa plataforma.
Mantenibilidad	4	363 líneas repartidas en 13 archivos, con una única responsabilidad por módulo y un tipo de resultado compartido (ResultadoSimulacion) que evita duplicar la capa de reporte entre algoritmos. Cada cabecera documenta su contrato en comentarios. Agregar un tercer algoritmo no requiere modificar Métricas ni Gantt.

Las dos calificaciones de 3 corresponden a limitaciones reales y verificables, no a modestia retórica: la validación incompleta del argumento de quantum y la falta de verificación en Windows son deficiencias concretas y corregibles.

8. Discusión de Resultados

8.1 Interpretación: qué algoritmo fue superior y bajo qué condiciones

FCFS resultó superior en tiempo de espera y retorno promedio en los dos datasets, con ventajas de 21.27 % y 28.27 % sobre RR $q=2$. La lectura ingenua de ese resultado — «FCFS es mejor» — no es defendible. La ventaja depende críticamente de la estructura de la carga: en ambos datasets las ráfagas son relativamente homogéneas (2–8 u.t. en el de referencia, 1–15 u.t. en el extendido) y las llegadas son densas y ordenadas, lo que impide que se manifieste un efecto convoy severo. Basta que un proceso de ráfaga muy larga llegue primero para que la relación se invierta.

Que la brecha se amplíe con el tamaño del dataset (de 21.27 % a 28.27 %) es coherente con el mecanismo del algoritmo: en Round Robin, el tiempo de espera de un proceso crece con el número de procesos activos en la cola durante su vida, porque debe esperar un turno de cada uno de ellos entre sus propios turnos. Con 50 procesos y ráfagas medias de 9.24 u.t. frente a un quantum de 2, cada proceso requiere en promedio cerca de cinco turnos, y el número total de turnos asignados asciende a 246 —4.92 veces los 50 de FCFS.

La comparación cambia de signo al desagregar por longitud de ráfaga. Round Robin $q=2$ mejoró la espera de los procesos cortos un 34.52 % y degradó la de los largos un 131.71 %, y redujo la desviación estándar de la espera un 15.50 %. Round Robin no es peor: optimiza un objetivo distinto —equidad y previsibilidad— al costo del promedio agregado.

8.2 Relación con la teoría

Tres observaciones del experimento admiten explicación teórica directa.

Convergencia hacia FCFS al crecer el quantum. Al pasar de $q=2$ a $q=4$, la espera de Round Robin se acercó a la de FCFS (6.33 % y 1.93 % de reducción) y los turnos cayeron 40.00 % y 44.72 %. Es exactamente lo previsto por la teoría [2], [3]: cuando el quantum supera la ráfaga máxima, ningún proceso es apropiado y Round Robin degenera en FCFS. Los valores observados son puntos intermedios de esa trayectoria.

FCFS con menor espera promedio. Ni FCFS ni Round Robin minimizan el tiempo de espera promedio; ese óptimo corresponde a Shortest Job First [1]. Lo observado no contradice la teoría sino que confirma un corolario menos citado: en ausencia de efecto convoy severo, la apropiación de Round Robin solo añade tiempo de espera, porque interrumpir un proceso para atender a otro traslada espera sin reducirla en el agregado.

Invariancia de la utilización de CPU. El hallazgo con explicación más limpia es que la utilización fue idéntica entre algoritmos: 100.00 % y 99.78 %. No es una coincidencia numérica. Ambos algoritmos son conservadores de trabajo, por lo que la CPU solo queda ociosa cuando no hay ningún proceso listo, condición que depende únicamente de los tiempos de llegada y no de la política. En consecuencia, el instante de finalización global es el mismo para ambos y la utilización se reduce a la razón entre la suma de ráfagas y ese intervalo. Los números lo confirman exactamente: en el dataset de referencia, 45 u.t. de ráfaga sobre 45 u.t. de

simulación produce 100.00 %; en el extendido, 462 sobre 463 produce 99.784 %, donde la única unidad ociosa es el intervalo entre $t=0$ y la llegada del primer proceso en $t=1$. La utilización de CPU, por tanto, no es una métrica útil para comparar planificadores conservadores de trabajo sin entrada/salida, y su inclusión aquí sirve como control de consistencia del simulador más que como criterio de comparación.

8.3 Limitaciones encontradas durante el desarrollo

- **El simulador no mide tiempo de respuesta.** Es la limitación más significativa, porque el tiempo de respuesta es la métrica en la que Round Robin exhibe su ventaja teórica frente a FCFS. Al reportar únicamente espera y retorno, el experimento mide a Round Robin con el criterio en el que estructuralmente pierde. Las métricas de dispersión de la Sección 6.5 son un sustituto parcial, no un reemplazo.
- **El costo del cambio de contexto se asume nulo,** lo que favorece artificialmente a Round Robin. Los 246 turnos de RR $q=2$ frente a 50 de FCFS en el dataset extendido son gratuitos en el modelo; con un costo realista de una unidad por conmutación, la desventaja de Round Robin sería considerablemente mayor.
- **Ausencia de operaciones de entrada/salida.** Sin bloqueos, la cola de listos casi nunca se vacía y la utilización de CPU permanece cerca del 100 %, lo que anula el poder discriminante de esa métrica (Sección 8.2). Un modelo con ráfagas de E/S intercaladas es la extensión de mayor valor experimental.
- **Sesgo de los datasets.** Ambos conjuntos fueron generados con ráfagas de rango acotado y llegadas casi consecutivas, un escenario favorable a FCFS. No se construyó un dataset adversarial con un proceso de ráfaga muy larga al inicio, que habría permitido observar el efecto convoy y contrastar la conclusión de la Sección 8.1.
- **Validación incompleta del argumento de quantum.** `std::stoi` sobre el segundo argumento no está protegido por `try/catch` ni se verifica que el valor sea positivo, a diferencia del rigor aplicado al archivo de entrada.
- **Compatibilidad con Windows no verificada** de forma directa; solo se argumenta por ausencia de dependencias de plataforma.

9. Conclusiones

El proyecto produjo un simulador funcional de planificación de CPU que compara FCFS y Round Robin sobre cargas controladas, con resultados deterministas y reproducibles a partir de datasets versionados. La contribución analítica no reside en el ranking de algoritmos sino en la caracterización de las condiciones bajo las cuales ese ranking es válido: FCFS aventajó a Round Robin en espera promedio entre 15.94 % y 28.27 % según el dataset y el quantum, pero exclusivamente porque las cargas evaluadas carecen de la dispersión de ráfagas necesaria para producir efecto convoy.

Tres aprendizajes concretos se derivan del desarrollo. El primero es que la elección de la métrica determina el resultado de la comparación tanto como el algoritmo: Round Robin pierde en espera promedio y gana en dispersión (−15.50 %) y en atención a procesos cortos (−34.52 % de espera), y no medir tiempo de respuesta ocultó su principal ventaja teórica. El segundo es que algunas métricas no discriminan en absoluto: la utilización de CPU resultó idéntica entre algoritmos por una razón estructural —la conservación de trabajo— y no por casualidad experimental, lo que enseña a distinguir entre una métrica de comparación y una métrica de control. El tercero es de ingeniería: definir un tipo de resultado común entre algoritmos permitió que la capa de reporte fuera escrita una sola vez, y esa decisión de diseño hizo que agregar el segundo algoritmo costara considerablemente menos que el primero.

Como extensiones fuera del alcance del proyecto se identifican, en orden de valor: incorporar el tiempo de respuesta como métrica reportada; modelar el costo del cambio de contexto como parámetro configurable, lo que volvería significativa la diferencia de 246 contra 50 turnos observada; construir un dataset adversarial que induzca efecto convoy para contrastar la conclusión sobre FCFS; e implementar Shortest Job First y planificación por prioridades para ampliar la comparación a un algoritmo que sí minimiza la espera promedio.

10. Referencias

- [1] A. Silberschatz, P. B. Galvin, y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2018.
- [2] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.^a ed. Boston, MA, EE. UU.: Pearson, 2015.
- [3] R. H. Arpaci-Dusseau y A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Madison, WI, EE. UU.: Arpaci-Dusseau Books, 2018.